

BEST-FIRST SEARCH

Aim:

To implement the Best-First Search algorithm using Prolog.

Objective:

To select the next node based on the smallest heuristic value and find a path toward the goal.

Algorithm:

1. Start with the initial node.
2. Add its successors to the OPEN list.
3. Select the node having the smallest heuristic value.
4. Remove the selected node from OPEN.
5. If it is the goal, stop.
6. Otherwise, expand its successors.
7. Repeat until the goal is found or OPEN becomes empty.

Flowchart

START

|

v

Add Start to OPEN

|

v

Select Node with Lowest $h(n)$

|

v

Is Goal?

/ \

Yes No

| |

STOP Expand Node

|

v

Add Children

|

v

Repeat

Prolog Program

% Best First Search

edge(a, b).

edge(a, c).

edge(b, d).

edge(b, e).

edge(c, f).

edge(d, g).

edge(e, g).

edge(f, g).

heuristic(a, 6).

heuristic(b, 4).

heuristic(c, 5).

heuristic(d, 2).

heuristic(e, 3).

heuristic(f, 2).

heuristic(g, 0).

best_first(Start, Goal, Path) :-

best_first_search([[Start]], Goal, Path).

```
best_first_search([[Goal|Rest]|_], Goal, Path) :-  
    reverse([Goal|Rest], Path).
```

```
best_first_search(Paths, Goal, Path) :-  
    select_best_path(Paths, BestPath),  
    remove_path(BestPath, Paths, Remaining),  
    BestPath = [Current|_],
```

```
    findall(  
        [Next|BestPath],  
        (  
            edge(Current, Next),  
            \+ member(Next, BestPath)  
        ),  
        NewPaths  
    ),
```

```
    append(Remaining, NewPaths, UpdatedPaths),
```

```
    best_first_search(UpdatedPaths, Goal, Path).
```

```
select_best_path([Path|Paths], Best) :-  
    path_heuristic(Path, H),  
    select_best(Path, H, Paths, Best).
```

```
select_best(Best, _, [], Best).
```

```
select_best(Current, H, [Path|Paths], Best) :-  
    path_heuristic(Path, H2),  
    (  
        H2 < H ->  
        select_best(Path, H2, Paths, Best)  
    ;  
        select_best(Current, H, Paths, Best)  
    ).
```

```
path_heuristic([Node|_], H) :-  
    heuristic(Node, H).
```

```
remove_path(Path, [Path|Rest], Rest).
```

```
remove_path(Path, [X|Rest], [X|NewRest]) :-  
    Path \= X,  
    remove_path(Path, Rest, NewRest).
```

Query

```
?- best_first(a, g, Path).
```

output: `Goal reached: g`

Result:

Thus, Best-First Search is successfully implemented using heuristic values.